EBIS Business Techschool

Tema 4 · Clase 1

Integración de modelos generativos vía API desde Python

LLM-as-a-Service: del prompt manual al componente software medible

Máster en Ingeniería de Automatización con IA Agéntica

www.ebiseducation.com



Clase 1 dentro del Tema 4

Hoy construimos la base técnica: invocar un modelo, medirlo y encapsularlo para automatización.

1. **APIs LLM Python**
2. Salida estructurada
3. Contexto + sesiones
4. FastAPI + streaming
5. LLMOps + despliegue

Resultado de hoy

Un cliente Python simple para llamar a un LLM, medir latencia y comparar configuraciones.

No entraremos aún

RAG, agentes, herramientas, bases vectoriales, despliegue cloud o MCP.

Tip

Entender el patrón antes de añadir complejidad.

Objetivos de aprendizaje de la sesión

Al terminar, el alumno debe poder explicar y programar una llamada LLM básica con criterio operativo.

01 **Entender LLM-as-a-Service**

Modelo externo invocado por API, con límites, coste y latencia.

02 **Construir una request**

API key, modelo, mensajes, parámetros y respuesta.

03 **Medir una llamada**

Latencia, longitud de entrada/salida y coste estimado.

04 **Diseñar una primera automatización**

Usar el modelo para resumir, clasificar o extraer información simple.

Caso práctico `llm_client.py` · `benchmark.py` · `main.py` · `config.yml` · `.env` · `results.csv`

El cambio clave: de chatbot a componente software

En producción, el LLM no “conversa”: presta una capacidad dentro de un sistema controlado.

Uso manual

- Usuario escribe un prompt
  Respuesta no necesariamente estructurada
  Sin métricas ni trazabilidad
  Difícil de repetir

Uso productivo

- Aplicación construye la request
  Modelo responde bajo requisitos
  Se validan resultados
  Se registran coste, latencia y errores
  Existe fallback

Automatización

El output alimenta un proceso posterior: clasificación, extracción, respuesta, alerta o revisión humana.

To sum up

Un sistema LLM debe diseñarse como una pieza de ingeniería: configurable, observable, repetible y mantenible.

EBIS Business Techschool

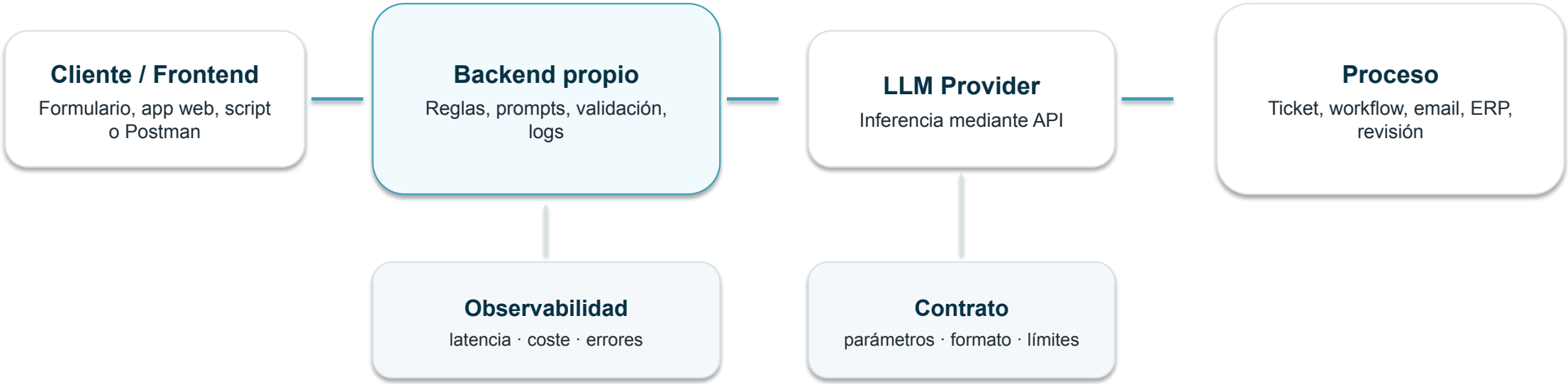
BLOQUE 1

1. Modelo generativo como servicio

Arquitectura, responsabilidades y trade-offs antes de escribir código.

Patrón base de una aplicación con LLM-as-a-Service

La aplicación conserva el control: el proveedor aporta inferencia, no gobierno del proceso.



Decisión de diseño

No enviamos “todo” al modelo: enviamos lo necesario, con parámetros controlados y salida procesable.

Proveedor, modelo y aplicación no son lo mismo

Separar estos conceptos evita decisiones incorrectas de arquitectura y coste.

Proveedor

- Empresa/plataforma que expone la API
- Gestiona infraestructura y seguridad del servicio
- Define autenticación, límites y facturación

Modelo

- Capacidad concreta seleccionada en la request
- Tiene trade-off de calidad, coste y latencia
- Puede cambiar de versión con impacto funcional

Aplicación

- Tu sistema de negocio
- Construye prompts, valida salidas y registra métricas
- Decide cuándo usar modelo y qué hacer con la respuesta

Regla práctica **Diseña tu código para poder cambiar de modelo sin reescribir toda la aplicación.**

Elegir modelo es una decisión de producto y operación

No hay un “mejor modelo” universal: hay un mejor ajuste para cada tarea y restricción.

Calidad
razonamiento, lenguaje, código

Latencia
tiempo hasta primera respuesta y total

Coste
precio por input/output token

Contexto
volumen máximo de información útil

Multimodalidad
texto, imagen, audio, documentos

Gobierno
región, privacidad, compliance

Caso	Prioridad dominante	Modelo típico
Chat interno simple	Coste / latencia	rápido y barato
Análisis legal largo	Contexto / precisión	modelo avanzado con contexto largo
Clasificación masiva	Coste / estabilidad	modelo pequeño + validación

EBIS Business Techschool

BLOQUE 2

2. Anatomía de una llamada LLM

Qué enviamos, qué recibimos y qué debemos medir.

Una llamada LLM tiene más partes que un prompt

La request combina credenciales, modelo, mensajes, parámetros y restricciones.

API key
identidad y permisos

Model
capacidad elegida

Messages
system / user / assistant

Parameters
temperature, max tokens

Constraints
timeout, safety, format

Response
texto, metadatos, usage

La calidad depende tanto del modelo como de la request.

En automatización, los parámetros y el formato de salida son parte del diseño, no un detalle técnico.

Los mensajes separan instrucciones, usuario e historial

Esta estructura será crítica en las siguientes clases para contexto, seguridad y salida estructurada.

Rol	Responsabilidad	Ejemplo
system	Comportamiento global y reglas permanentes	“You are a process automation assistant…”
user	Petición o dato enviado por el usuario/aplicación	“Clasifica este ticket…”
assistant	Respuestas previas si hay conversación	“Necesito el número de pedido.”
tool/function	Resultados externos, si el proveedor lo soporta	“order_status=delivered”

Clase 1 Hoy usamos un patrón mínimo. En Clase 3 profundizaremos en historial, contexto y sesiones.

Los parámetros convierten el modelo en un componente controlable

Para automatización buscamos estabilidad, límite de salida y comportamiento predecible.

Parámetro	Qué controla	Recomendación inicial
model	Capacidad, precio y latencia	empezar con uno equilibrado; escalar si falla
temperature	Variabilidad de la generación	0.0–0.3 para extracción/clasificación
max_tokens	Tamaño máximo de salida	limitar para controlar coste y latencia
top_p / top_k	Muestreo alternativo	no tocar al inicio salvo razón clara
timeout	Tiempo máximo de espera	siempre definido en aplicaciones
stream	Entrega incremental de tokens	útil para UX; no siempre para salidas estructuradas

Temperatura: creatividad vs consistencia

Una configuración “creativa” puede ser mala para automatizar procesos repetibles.



Para clasificación / extracción

- temperature baja
- max_tokens limitado
- validación posterior

Para ideación / redacción

- temperature media
- más libertad de salida
- revisión humana esperable

Tokens, coste y latencia son variables de diseño

Cada llamada tiene un coste aproximado: input + output + overhead del proveedor.

Fórmula mental de coste

≈

input_tokens × precio_input

+

output_tokens × precio_output

Prompt
instrucciones + datos

Modelo
inferencia

Output
tokens generados

Sube el coste cuando…

- añades más contexto
- pides respuestas largas
- repites llamadas en batch

Baja el coste cuando…

- resumes inputs
- limitas max_tokens
- usas modelos pequeños/eficientes

Regla práctica

- mide antes de optimizar
- compara por tarea
- no optimices a ciegas

La latencia tiene varios causantes

Medir solo el tiempo total oculta dónde está el problema.

Network
ida/vuelta API

Queue
carga proveedor

Context
tokens de entrada

Generation
tokens de salida

Validation
postproceso propio

Tiempo hasta primera respuesta

Importante para experiencia de usuario; mejora con streaming.

Tiempo total de tarea

Importante para automatización batch y procesos síncronos.

Errores frecuentes al integrar LLMs por API

Los fallos esperables deben formar parte del diseño desde el día uno.

401 / Auth
clave incorrecta o sin permisos

429 / Rate limit
demasiadas peticiones

Timeout
el modelo no responde a tiempo

Context length
input demasiado largo

Invalid output
formato no procesable

Safety refusal
respuesta bloqueada o parcial

Principio operativo

Toda llamada externa necesita timeout, gestión de errores y logging mínimo.

Patrón mínimo de resiliencia para una llamada LLM

No buscamos perfección: buscamos fallar de forma controlada y observable.

1. **Timeout**
   Evita que el proceso quede bloqueado.

2. **Retry + backoff**
   Reintenta fallos transitorios sin saturar.

3. **Fallback**
   Cambia a modelo alternativo o ruta manual.

4. **Log**
   Registra input resumido, modelo, latencia, error.

5. **Validation**
   Comprueba que la salida sirve para el proceso.

Clase 2 La validación de salida estructurada se trabajará en profundidad con JSON/Pydantic.

EBIS Business Techschool

BLOQUE 3

3. Demo y práctica guiada

Construimos un cliente Python medible para automatizar tareas simples.

Demo: de cero a primera llamada medible

1. **Configurar entorno**
   .env + requirements
2. **Crear cliente**
   función call_llm()
3. **Enviar prompt**
   resumen / clasificación
4. **Medir latencia**
   perf_counter()
5. **Guardar resultado**
   CSV o JSONL

Resultado esperado

No solo se “obtiene una respuesta”; se obtiene una respuesta con metadatos mínimos para decidir si el modelo es usable.

Código base: cliente LLM con medición de latencia

La función encapsula la llamada y devuelve una estructura que podremos evaluar más adelante.

llm_client.py

```python
from dataclasses import dataclass
from time import perf_counter


@dataclass
class LLMResult:
    text: str
    model: str
    latency_ms: int
    input_tokens: int
    output_tokens: int
    input_chars: int
    output_chars: int
```

```python
def call_llm(
    client,
    prompt: str,
    model: str,
    temperature: float = 0.2,
    max_tokens: int = 500
) -> LLMResult:
    start = perf_counter()

    response = client.messages.create(
        model=model,
        max_tokens=max_tokens,
        temperature=temperature,
        messages=[
            # TODO 1: add the user message with the prompt
        ],
    )

    # TODO 2: extract the text from the response
    text = ...

    latency_ms = int((perf_counter() - start) * 1000)

    return LLMResult(
        text=text,
        model=model,
        latency_ms=latency_ms,
        input_tokens=...,   # TODO 3: read input tokens from response usage
        output_tokens=...,  # TODO 4: read output tokens from response usage
        input_chars=len(prompt),
        output_chars=len(text),
    )
```

Configuración: las claves nunca van en el código

Separar configuración de lógica permite seguridad, cambios de modelo y despliegues más limpios.

.env.example

```
# .env.example

LLM_API_KEY=replace_with_your_api_key
LLM_MODEL=claude-haiku-4-5-20251001
LLM_TIMEOUT_SECONDS=30
```

main.py

```
# main.py
from llm_client import call_llm

prompt = "Summarize the following customer request..."
result = call_llm(client, prompt, model="selected-model")

print(result.text)
print(result.latency_ms)
```

Buenas prácticas

- usar variables de entorno
- no subir .env al repositorio
- crear .env.example

Riesgos

- API key expuesta
- coste no controlado
- difícil rotación de claves

Objetivo

- cambiar proveedor sin tocar lógica
- facilitar despliegue posterior

Caso práctico: AI Request Triage

Usaremos solicitudes de cliente para practicar resumen, clasificación y extracción simple.

Input

Texto libre escrito por un cliente o usuario interno.

Ejemplo:
“El cliente quiere cancelar el pedido 8921 porque llegó roto y solicita reembolso urgente.”

LLM call

Prompt + modelo + parámetros

temperature=0.2
max_tokens=250

Output inicial

- Resumen
  Tipo de solicitud
  Entidades simples
  Acción sugerida

En Clase 2 lo convertiremos en JSON validado.

Foco Medir y comparar llamadas; todavía no exigimos salida perfectamente estructurada.

Ejercicio de la clase: tres entregables incrementales

El alumno empieza con una llamada aislada y termina con una comparación simple de configuraciones.

A — Primera llamada

Implementa call_llm() y ejecuta un prompt de resumen.

B — Batch de solicitudes

Ejecuta el modelo sobre una lista de tickets y guarda resultados.

C — Comparación

Compara dos temperaturas o dos modelos: salida, latencia y coste aproximado.

Métrica	Cómo medirla hoy	Para qué sirve
latency_ms	perf_counter()	comparar experiencia y batch
input/output length	caracteres o tokens estimados	aproximar coste
output usefulness	revisión manual rápida	preparar evaluación futura

Dataset: solicitudes cortas de cliente

ID	Canal	Tarea	Solicitud del cliente	Categoría esperada	Prioridad
REQ-001	email	classify	The customer wants to cancel order 8921 because the item arrived broken.	`refund_or_cancellation`	medium
REQ-002	chat	classify	I need to change the delivery address for order 5520 before it ships.	`delivery_change`	medium
REQ-003	email	classify	The user reports that they have not been able to access the platform since yesterday.	`technical_support`	medium
REQ-004	form	summarize	Please summarize this complaint: the invoice amount is wrong and the client has already contacted support twice.	`billing_issue`	high

Estructura recomendada del repositorio de la clase

Una organización simple facilita mantener el proyecto durante las cinco sesiones.

Project structure

```
llm_api_demo/
│
├── .env
├── config.yml
├── llm_client.py
├── main.py
├── benchmark.py
│
├── data/
│   └── customer_requests_demo.csv
│
└── outputs/
    └── benchmark_results.csv
```

Por qué así

- separa datos, lógica y resultados
- prepara las siguientes clases
- evita notebooks desordenados

Salida de la práctica

- results.csv con respuesta y latencia
- comparación manual de configuraciones
- cliente reutilizable

Siguiente evolución

- prompt templates y JSON validado en Clase 2

Código de benchmark simple

No hace falta un framework complejo para empezar a operar con criterio.

benchmark.py

```
import os
from pathlib import Path

import pandas as pd
import yaml
from anthropic import Anthropic
from dotenv import load_dotenv

from llm_client import call_llm


with open("config.yml", "r", encoding="utf-8") as f:
    config = yaml.safe_load(f)

load_dotenv()

client = Anthropic(
    api_key=os.getenv("LLM_API_KEY"),
    timeout=int(os.getenv("LLM_TIMEOUT_SECONDS", 30)),
)

model = os.getenv("LLM_MODEL")

df = pd.read_csv(config["input_csv"])

results = []
```

```
for _, row in df.iterrows():
    prompt = f"""
{config["prompt"]["role"]}

Task:
{row["task"]}

Customer request:
{row["customer_request"]}

Instructions:
{config["prompt"]["instructions"]}

Output format:
{config["prompt"]["output_format"]}
"""

    print(f'Processing {row["request_id"]}...')

    result = call_llm(
        client=client,
        prompt=prompt,
        model=model,
        temperature=...,  # TODO 1: read from config
        max_tokens=...,   # TODO 2: read from config
    )

    results.append({
        "request_id": row["request_id"],
        "task": row["task"],
        "customer_request": row["customer_request"],
        "model": result.model,
        "latency_ms": result.latency_ms,
        "input_tokens": ...,   # TODO 3: add input tokens
        "output_tokens": ...,  # TODO 4: add output tokens
        "status": "ok",
        "output": ...,         # TODO 5: add model response
    })

output_path = Path(config["output_csv"])
output_path.parent.mkdir(parents=True, exist_ok=True)

pd.DataFrame(results).to_csv(output_path, index=False, encoding="utf-8")

print(f"Benchmark completed: {output_path}")
```

EBIS Business Techschool

BLOQUE 4

4. Debrief y cierre

Qué debe quedar claro antes de pasar a prompting estructurado.

Errores habituales en la primera integración

Estos errores son simples, pero en producción generan coste, riesgo y comportamiento inestable.

Clave en código
riesgo de seguridad y coste

Sin timeout
procesos bloqueados

Prompt gigante
latencia y coste innecesarios

Temperatura alta
salidas poco repetibles

Sin logging
imposible depurar

Sin separación por capas
difícil cambiar de proveedor

Mini-quiz de cierre

1. ¿Por qué no debemos hardcodear una API key?
2. ¿Qué parámetro reducirías para una tarea de clasificación estable?
3. ¿Qué dos variables explican el coste de una llamada?
4. ¿Qué harías ante un error 429?
5. ¿Por qué conviene encapsular la llamada en llm_client.py?

El patrón mínimo

Clase 2: una automatización más fiable mediante prompts y salidas estructuradas.

Hoy

Llamada LLM desde Python
Parámetros básicos
Medición de latencia
Comparación simple

Clase 2

Prompt engineering operativo
Salida estructurada
Pydantic
Retry si no valida

Meta

Automatizar procesos con outputs procesables y controlados.

To sum up

Primero hacemos que el modelo sea invocable y medible. Después haremos que sea fiable.

Material de apoyo y checklist

Para preparar la práctica, basta con tener entorno Python y una API key del proveedor seleccionado.

Checklist técnico

- Python
- entorno virtual creado
- paquete SDK del proveedor
- .env configurado
- carpeta outputs/ creada

Lecturas recomendadas

- documentación oficial del proveedor usado (anthropic)
- guías de pricing del proveedor (anthropic)
- guías de seguridad para API keys

Nota de diseño

- Los precios, modelos disponibles y límites de rate limit cambian con frecuencia. En clase se usará la documentación vigente del proveedor elegido.